Code Explanation

Batch Processing Code Explanation

This code is designed to process customer and order data using Apache Spark. It reads data from CSV files, cleans and processes it, and then updates two Delta tables: `ordersummary` and `customeraggregatespend`.

Overview
The code is structured into several functions, each responsible for a specific task, such as creating a Spark session, reading data, cleaning data, processing order data, and updating Delta tables. The `main` function orchestrates the entire data processing pipeline.

Step 1: Creating a Spark Session
This step creates a Spark session with Hive support enabled.

```
def create_spark_session():
    return SparkSession.builder
        .appName("Customer Order Processing")
        .enableHiveSupport()
        .getOrCreate()
```

Step 2: Reading Customer and Order Data
These steps read customer and order data from CSV files stored in volumes. The data is read into DataFrames using the `spark.read.csv` method with a predefined schema.

```
def read_customer_data(spark):
    customer_path = "/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/c
ustomerdata"
    customer_schema = StructType([
        StructField("CustId", StringType(), True),
        StructField("Name", StringType(), True),
        StructField("EmailId", StringType(), True),
        StructField("Region", StringType(), True)
    ])
    return spark.read.csv(customer_path, header=True, schema=custome
r_schema)

def read_order_data(spark):
    order_path = "/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orde
rdata"
    order_schema = StructType([
        StructField("OrderId", StringType(), True),
        StructField("ItemName", StringType(), True),
        StructField("PricePerUnit", DoubleType(), True),
        StructField("Qty", IntegerType(), True),
        StructField("Date", DateType(), True),
        StructField("CustId", StringType(), True)
    ])
    return spark.read.csv(order_path, header=True, schema=order_sche
ma)
```

Step 3: Cleaning Data
This step removes null values and duplicates from the customer and order DataFrames.

```
def clean_data(df):
    df_no_nulls = df.dropna()
    df_clean = df_no_nulls.dropDuplicates()
    return df_clean
```

Step 4: Processing Order Data
This step adds a new column `TotalAmount` to the order DataFrame by multiplying `PricePerUnit` and `Qty`.

```
def process_order_data(order_df):
    return order_df.withColumn("TotalAmount", col("PricePerUnit") *
col("Qty"))
```

Step 5: Creating Catalog and Schema
This step creates a catalog and schema if they don't exist.

```
def create_catalog_schema(spark):
    spark.sql("CREATE CATALOG IF NOT EXISTS gen_ai_poc_databrickscoe
")
    spark.sql("CREATE SCHEMA IF NOT EXISTS gen_ai_poc_databrickscoe.
sdlc_wizard")
```

Step 6: Creating Delta Tables
These steps create two Delta tables: `ordersummary` and `customeraggregatespend` if they don't exist.

```
def create_ordersummary_table(spark):
    spark.sql("""
    CREATE TABLE IF NOT EXISTS gen_ai_poc_databrickscoe.sdlc_wizard.
ordersummary (
        CustId STRING,
        Name STRING,
        EmailId STRING,
        Region STRING,
        OrderId STRING,
        ItemName STRING,
        PricePerUnit DOUBLE,
        Qty INT,
        Date DATE,
        TotalAmount DOUBLE,
        IsActive BOOLEAN,
        StartDate TIMESTAMP,
        EndDate TIMESTAMP
    )
    USING DELTA
    """)

def create_customeraggregatespend_table(spark):
    spark.sql("""
    CREATE TABLE IF NOT EXISTS gen_ai_poc_databrickscoe.sdlc_wizard.
customeraggregatespend (
        Name STRING,
        TotalAmount DOUBLE,
        Date DATE
    )
    USING DELTA
```

```
    """)
```

Step 7: Updating SCD Type 2 Table
This step updates the `ordersummary` Delta table using SCD Type 2 logic. It identifies changed records, marks them as inactive, and inserts new records.

```
def update_scd_type2_table(spark, customer_df, order_df):
    # Get current data from ordersummary
    current_data = spark.table("gen_ai_poc_databrickscoe.sdlc_wizard
.ordersummary")

    # Join customer and order data
    joined_data = customer_df.join(order_df, "CustId")

    # Prepare new data with SCD Type 2 columns
    new_data = joined_data.select(
        "CustId", "Name", "EmailId", "Region", "OrderId", "ItemName"
,
        "PricePerUnit", "Qty", "Date", "TotalAmount"
    ).withColumn("IsActive", lit(True))
     .withColumn("StartDate", current_timestamp())
     .withColumn("EndDate", lit(None).cast("timestamp"))

    # ... (rest of the implementation)
```

Step 8: Updating Customer Aggregate Spend Table
This step aggregates the `TotalAmount` by `Name` and `Date` from the `ordersummary` table and updates the `customeraggregatespend` Delta table.

```
def update_customeraggregatespend(spark):
    active_records = spark.table("gen_ai_poc_databrickscoe.sdlc_wiza
rd.ordersummary")
                          .filter(col("IsActive") == True)

    aggregated_data = active_records.groupBy("Name", "Date")
                                    .agg(sum_("TotalAmount").alias("T
otalAmount"))

    aggregated_data.write.format("delta").mode("overwrite")
        .saveAsTable("gen_ai_poc_databrickscoe.sdlc_wizard.customera
ggregatespend")
```

Step 9: Orchestrating the Data Processing Pipeline
The `main` function calls the above steps in sequence to process the customer and order data.

```
def main():
    spark = create_spark_session()

    # Read data from volumes
    customer_df_raw = read_customer_data(spark)
    order_df_raw = read_order_data(spark)

    # ... (rest of the implementation)
```